

ABC PHARMACY - SALESFORCE ENTRY ASSESSMENT

ABC Pharmacy REST API

Reference for the Ionic application developers

VERSION	1.0
DATE	14 September 2026
AUTHOR	Youssef Maged — prepared for Cloudastick Systems
STATUS	Issued

Contents

1. 1. Base URL

2. 2. Authentication

3. 3. The response envelope

4. 4. Error codes

5. 5. Endpoints

5.1 GET /abc/v1/products

5.2 POST /abc/v1/carts

5.3 GET /abc/v1/carts/{cartId}

5.4 POST /abc/v1/carts/{cartId}/items

5.5 POST /abc/v1/carts/{cartId}/checkout

6. 6. Notes for the client

Everything the Ionic app needs to browse the ABC catalogue and build a shopping cart. A Postman collection covering every call in this document is in `postman/`.

1. Base URL

```
https://orgfarm-ad027e59fe-dev-ed.develop.my.salesforce.com/services/apexrest
```

All paths below are relative to that. Use **your org's My Domain host** — not `login.salesforce.com`, which will not serve these endpoints.

Every request and response is `application/json; charset=UTF-8`.

Verified against the org. Every request and response in this document was captured from a live call to `orgfarm-ad027e59fe-dev-ed.develop.my.salesforce.com`. The accompanying Postman collection runs 23 requests and 119 assertions against these endpoints with no failures.

2. Authentication

The API is registered as an External Client App called `abc_Ionic_App` using the OAuth 2.0 **client credentials** flow. Request a token, then send it as a bearer token.

Request

```
POST /services/oauth2/token HTTP/1.1
Host: orgfarm-ad027e59fe-dev-ed.develop.my.salesforce.com
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=<consumer key>
&client_secret=<consumer secret>
```

Response

```
{
  "access_token": "00Dg8000...!AQEAQJ3x...",
  "instance_url": "https://orgfarm-ad027e59fe-dev-ed.develop.my.salesforce.com",
  "token_type": "Bearer",
  "issued_at": "1789301234567"
}
```

Use it

```
GET /services/apexrest/abc/v1/products HTTP/1.1
Authorization: Bearer 00Dg8000...!AQEAQJ3x...
```

Tokens expire according to the org's session policy. Treat a `401` as "fetch a new token and retry once".

Production note. Client credentials authenticates the *application*, not the person using it: every order created this way is attributed to one shared integration user. It is used here because it keeps a reviewer's setup to a single call. A production Ionic app should use **Authorization Code with PKCE**, so each customer acts as themselves, or route calls through a backend that holds the secret. Never ship a client secret inside a mobile binary.

3. The response envelope

Every response uses the same four keys, whether it succeeded or failed. Parse one shape; never branch on the status code to find the payload.

```
{
  "success": true,
  "data": { },
  "errors": [],
  "requestId": "4Zx9KpQ2mN"
}
```

Key	Type	Notes
<code>success</code>	boolean	
<code>data</code>	object or <code>null</code>	Always present. <code>null</code> on failure.
<code>errors</code>	array	Always present. Empty on success.
<code>errors[].code</code>	string	Stable machine-readable code — branch on this, not on the message.
<code>errors[].message</code>	string	Human readable. Wording may change; the code will not.
<code>errors[].field</code>	string	Present only when one request field is at fault.
<code>requestId</code>	string	Correlation id, also stamped on every server-side log entry for this request.

Always log `requestId`. Quoting it in a support request lets the Salesforce team find the exact server-side log rows for that call.

A failure looks like this:

```
{
  "success": false,
  "data": null,
  "errors": [
    {
      "code": "INSUFFICIENT_STOCK",
      "message": "Only 2 unit(s) of \"Fingertip Pulse Oximeter\" are in stock; the cart would hold 5.",
      "field": "quantity"
    }
  ],
  "requestId": "4Zx9KpQ2mN"
}
```

4. Error codes

Code	HT TP	Meaning	What the app should do
<code>INVALID_INPUT</code>	400	A required value is missing, or a query parameter is not the right type.	Fix the request. Check <code>field</code> .
<code>MALFORMED_JSON</code>	400	The body is not valid JSON, has the wrong shape, or contains an attribute this endpoint does not define.	Fix the request. Usually a typo in a field name.
<code>VALIDATION_ERROR</code>	400	The request parsed, but a business rule rejected the values.	Show the message; check <code>field</code> .
<code>CART_NOT_FOUND</code>	404	No cart with that id.	Start a new cart.
<code>PRODUCT_NOT_FOUND</code>	404	No such product, or it is not sellable.	Refresh the catalogue.
<code>ROUTE_NOT_FOUND</code>	404	No endpoint at that path.	A client bug — the message lists the valid routes.
<code>CART_NOT_EDITABLE</code>	409	The cart has been checked out and its contents are frozen.	Start a new cart.
<code>INSUFFICIENT_STOCK</code>	409	The warehouse cannot supply that many units.	Show the message; offer the available quantity.
<code>INTERNAL_ERROR</code>	500	Something went wrong server-side.	Retry once; if it persists, report the <code>requestId</code> .

`INTERNAL_ERROR` never carries detail — the exception, message and stack trace are recorded server-side against the same `requestId`.

5. Endpoints

5.1 `GET /abc/v1/products`

The catalogue.

What counts as the catalogue. A product is returned when it is active *and* has an active entry in the **standard price book** — that pairing is what makes a product sellable, and it is the only definition the service uses. It deliberately does not filter on the `ABC-` product code, so adding a new pharmacy line needs no code change. The consequence is that any other active, priced product in the org would also be listed: a Developer Edition org ships with a sample catalogue (GenWatt generators, SLAs, Installations), and `scripts/apex/abc_deactivate_sample_products.apex` deactivates it. Run that script in any fresh org, or those records will appear here too.

Query parameters — all optional.

Name	Type	Default	Notes
<code>search</code>	string	—	Substring, case-insensitive, matched against product name and product code.
<code>category</code>	string	—	One of <code>Medicine</code> , <code>Supplement</code> , <code>Personal Care</code> , <code>Medical Device</code> .
<code>includeOutOfStock</code>	boolean	<code>false</code>	Only <code>true</code> or <code>false</code> ; anything else is a 400.
<code>limit</code>	integer	<code>50</code>	1–200. Larger values are clamped to 200.
<code>offset</code>	integer	<code>0</code>	Rows to skip. Capped at 2000.

Example

```
GET /services/apexrest/abc/v1/products?category=Medicine&search=para&limit=2
Authorization: Bearer <token>
```

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "01tg80000063Y1hAAE",
        "name": "Paracetamol 500mg Tablets (20)",
        "productCode": "ABC-MED-001",
        "description": "Analgesic and antipyretic tablets, blister pack of 20.",
        "category": "Medicine",
        "requiresPrescription": false,
        "stockQuantity": 500,
        "unitPrice": 12.50,
        "currencyIsoCode": "USD"
      }
    ],
    "count": 1,
    "limit": 1,
    "offset": 0
  },
  "errors": [],
  "requestId": "SLB:dc1ea05a428b5820bf29acb69228895e"
}
```

`count` is the number of items in **this page**, not the size of the catalogue. Page until a response returns fewer than `limit` items.

A product is only listed when it is active and has an active price in the standard price book. Out-of-stock products are hidden unless `includeOutOfStock=true` ; use that on a "notify me" screen rather than the main list.

5.2 `POST /abc/v1/carts`

Opens a cart. A cart is a Draft order — it is a real record from the moment it is created, so the pharmacy can see abandoned carts.

The customer is matched by **email**. If a contact with that address exists it is reused; otherwise an account named `"First Last"` and its contact are created.

Body

Field	Type	Required	Notes
customer.email	string	yes	Must be a valid address. This is the customer's identity.
customer.lastName	string	yes	
customer.firstName	string	no	
customer.phone	string	no	
items	array	no	Up to 100 entries. Repeats of the same product are merged.
items[].productId	string	yes	Salesforce Product2 id (18 characters, starts 01t).
items[].quantity	number	yes	Whole number, 1–9999.

```
POST /services/apexrest/abc/v1/carts
Authorization: Bearer <token>
Content-Type: application/json

{
  "customer": {
    "email": "mona.hassan@example.com",
    "firstName": "Mona",
    "lastName": "Hassan",
    "phone": "+20 100 111 2222"
  },
  "items": [
    { "productId": "01tg8000004xYzAAAU", "quantity": 2 }
  ]
}
```

201 Created

```
{
  "success": true,
  "data": {
    "cartId": "801g800000nSwpdAAC",
    "orderNumber": "00000115",
    "status": "Draft",
    "customer": {
      "accountId": "001g800000nSwo1AAC",
      "contactId": "003g800000inTdpAAE",
      "email": "doc.example@example.com",
      "name": "Doc Example"
    },
    "items": [
      {
        "itemId": "802g800000BIrh3AAD",
        "productId": "01tg80000063Y1hAAE",
        "productName": "Paracetamol 500mg Tablets (20)",
        "quantity": 2.0,
        "unitPrice": 12.50,
        "totalPrice": 25.00
      }
    ],
    "totalAmount": 25.00,
    "createdDate": "2026-09-13T13:59:07Z"
  },
  "errors": [],
  "requestId": "SLB:d9443032c0af374c50628583e08eb8ca"
}
```

Every cart response uses this same `CartDto` shape.

Failures: `INVALID_INPUT` (missing or invalid email, missing last name), `PRODUCT_NOT_FOUND`, `VALIDATION_ERROR` (bad quantity), `INSUFFICIENT_STOCK`, `MALFORMED_JSON`.

5.3 `GET /abc/v1/carts/{cartId}`

Reads a cart. Returns 200 with the same `CartDto`.

```
GET /services/apexrest/abc/v1/carts/801g8000003xAbCAAU
Authorization: Bearer <token>
```

Failures: `CART_NOT_FOUND` (unknown, malformed, or an id belonging to another object).

5.4 `POST /abc/v1/carts/{cartId}/items`

Adds a product. If the product is already in the cart the existing line's quantity is increased — a cart always holds one line per product, so the app does not have to check first.

Body

Field	Type	Required	Notes
productId	string	yes	
quantity	number	yes	Whole number, 1–9999. This is the amount to add .

```
POST /services/apexrest/abc/v1/carts/801g8000003xAbCAAU/items
Authorization: Bearer <token>
Content-Type: application/json

{ "productId": "01tg8000004xYzAAAU", "quantity": 3 }
```

Returns **200** with the updated cart. Adding 3 of a product already in the cart at quantity 2 yields one line at 5, not two lines:

```
{
  "success": true,
  "data": {
    "cartId": "801g800000nSwpdAAC",
    "orderNumber": "00000115",
    "status": "Draft",
    "items": [
      {
        "itemId": "802g800000BIrh3AAD",
        "productId": "01tg80000063Y1hAAE",
        "productName": "Paracetamol 500mg Tablets (20)",
        "quantity": 5.0,
        "unitPrice": 12.50,
        "totalPrice": 62.50
      }
    ],
    "totalAmount": 62.50
  },
  "errors": [],
  "requestId": "SLB:53355930e93c929e1ba2f0c81b22c582"
}
```

Note the `itemId` is unchanged - the existing line was raised rather than a new one created.

Stock is checked against what the cart would hold **after** the merge, so adding 2 then 4 of a product with 5 in stock fails on the second call.

Failures: `CART_NOT_FOUND`, `PRODUCT_NOT_FOUND`, `VALIDATION_ERROR`, `INSUFFICIENT_STOCK`, `CART_NOT_EDITABLE`, `MALFORMED_JSON`.

5.5 POST /abc/v1/carts/{cartId}/checkout

Places the order: the same record moves from `Draft` to `Activated`, and its order date is set to today. No body; anything sent is ignored, so a retry with a stale payload cannot change the order.

```
POST /services/apexrest/abc/v1/carts/801g8000003xAbCAAU/checkout
Authorization: Bearer <token>
```

Returns **200** with the cart, now `"status": "Activated"` - and the same `cartId`, because checkout activates the existing order rather than creating a second record.

A further attempt to change it returns:

```
{
  "success": false,
  "data": null,
  "errors": [
    {
      "code": "CART_NOT_EDITABLE",
      "message": "Cart 00000115 has status \"Activated\" and can no longer be changed.",
      "field": "status"
    }
  ],
  "requestId": "SLB:172125ba1e70380f3db1938425402227"
}
```

After checkout the order is frozen — further `items` calls return `CART_NOT_EDITABLE`. From here the warehouse moves it to `In delivery` and then `Delivered`.

Failures: `CART_NOT_FOUND`, `CART_NOT_EDITABLE` (already checked out), `VALIDATION_ERROR` (the cart is empty).

6. Notes for the client

Strict bodies. Request bodies are parsed strictly: an attribute that is not part of the documented body is rejected with `MALFORMED_JSON`. This is deliberate — a misspelled field fails loudly at the call site instead of being read as absent and producing a confusing error later. Send exactly the fields documented above.

Ids. `cartId` is a Salesforce Order id (starts `801`), `productId` a `Product2` id (starts `01t`), `itemId` an `OrderItem` id (starts `802`). Treat them as opaque 18-character strings.

Quantities are whole numbers. `1.5` is rejected with `VALIDATION_ERROR`.

Money is returned as a JSON number with two decimal places, in the org's currency, given per product as `currencyIsoCode`.

Recommended flow

GET /abc/v1/products?limit=50&offset=0	browse, page as the user scrolls
POST /abc/v1/carts	on first add – keep the cartId
POST /abc/v1/carts/{cartId}/items	each subsequent add
GET /abc/v1/carts/{cartId}	to re-render the cart screen
POST /abc/v1/carts/{cartId}/checkout	place the order

Keep `cartId` for the life of the session. A cart left in Draft stays valid indefinitely and can be resumed.

Retries. `GET` calls are safe to retry. `POST /items` is **not** idempotent — retrying adds the quantity again — so retry it only after a network failure with no response, and re-read the cart afterwards to confirm. Checkout is safe to retry: a second call returns `CART_NOT_EDITABLE`, which tells you the first one succeeded.